Neural Networks Design and Deployment

Group Members :

- **202301040062 — Pratik Bhopi**
- **202301040133 — Krishna Rane**
- **202301040159 — Kartik Mandake**
- **202301040202 — Pranav Paralkar**

Inputs :
x1 = 2
x2 = 2

Input → Hidden

w11 = 0.3 (x1→h1)
w21 = -0.5 (x2→h1)
w12 = 0.6 (x1→h2)
w22 = 0.4 (x2→h2)

Hidden → Output
v1 = -0.2 (h1→y)
v2 = 0.8 (h2→y)

Target = 1.5
Learning rate = 0.1

```
import numpy as np

# Sigmoid Activation Function
# It maps any input to a value between 0 and 1.
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of Sigmoid Activation Function
# This is used during backpropagation to calculate the gradient.
def sigmoid_derivative(x):
    return x * (1 - x)


class NeuralNetwork:

  # - input_size: Number of input features
  # - hidden_size: Number of neurons in the hidden layer
  # - output_size: Number of output neurons

    def __init__(self, input_size, hidden_size, output_size):

        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size

        # w11=0.3, w12=0.6
        # w21=-0.5, w22=0.4
        self.weights_input_hidden = np.array([
            [0.3, 0.6],
            [-0.5, 0.4]
        ])

        # v1=-0.2, v2=0.8
        self.weights_hidden_output = np.array([
            [-0.2],
            [0.8]
        ])

        #  No bias in diagram
        self.bias_hidden = np.zeros((1, self.hidden_size))
        self.bias_output = np.zeros((1, self.output_size))
```

Overall, how satisfied are you with Colaboratory?

Very satisfied

Somewhat satisfied

Neither satisfied nor dissatisfied

Somewhat dissatisfied

Very dissatisfied

Next

By continuing, you agree Google uses your answers, account & system info to improve services, per our Privacy & Terms.

```
    def forward(self, X):

        self.input_layer = X #Input data{features}

        # Calculate the input to the hidden layer and apply the activation function
        self.hidden_layer_input = np.dot(self.input_layer, self.weights_input_hidden) + self.bias_hidden
        self.hidden_layer_output = sigmoid(self.hidden_layer_input)

        # Calculate the input to the output layer and apply the activation function
        self.output_layer_input = np.dot(self.hidden_layer_output, self.weights_hidden_output) + self.bias_output
        self.output_layer_output = sigmoid(self.output_layer_input)

        print("\n--- Forward Pass ---")
        print("Hidden linear:", self.hidden_layer_input)
        print("Hidden activation:", self.hidden_layer_output)
        print("Output linear:", self.output_layer_input)
        print("Output activation:", self.output_layer_output)

        return self.output_layer_output

    def backward(self, X, y, learning_rate):

      # - X: Input data (features)
      # - y: True labels (targets)

        print("\n--- Backward Pass ---")

        error_output = y - self.output_layer_output
        print("Output error:", error_output)

        output_layer_delta = error_output * sigmoid_derivative(self.output_layer_output)
        print("Output delta:", output_layer_delta)

        error_hidden = output_layer_delta.dot(self.weights_hidden_output.T)
        hidden_layer_delta = error_hidden * sigmoid_derivative(self.hidden_layer_output)

        print("Hidden delta:", hidden_layer_delta)

        # weight updates
        self.weights_hidden_output += self.hidden_layer_output.T.dot(output_layer_delta) * learning_rate
        self.weights_input_hidden += X.T.dot(hidden_layer_delta) * learning_rate

        print("\nUpdated W_hidden_output:\n", self.weights_hidden_output)
        print("Updated W_input_hidden:\n", self.weights_input_hidden)


# ========================
# Main — diagram problem
# ========================

if __name__ == "__main__":

    # Diagram inputs
    X = np.array([[2, 2]])
    y = np.array([[1.5]])

    # 2 hidden neurons
    nn = NeuralNetwork(input_size=2, hidden_size=2, output_size=1)

    learning_rate = 0.1

    # Only ONE step to match manual solution
    output_before = nn.forward(X)
    nn.backward(X, y, learning_rate)
    output_after = nn.forward(X)

    print("\nOutput before update:", output_before)
    print("Output after update:", output_after)



    # ========================
    # Extra training iterations
    # ========================

    print("\n--- Continuing Training Iterations ---")
```

```
epochs = 500

for i in range(epochs):
    nn.forward(X)
    nn.backward(X, y, learning_rate)

    if i % 50 == 0:
        out = nn.forward(X)
        loss = np.mean((y - out) ** 2)
        print(f"Iter {i:4d} | Output = {out[0][0]:.6f} | Loss = {loss:.6f}")

final_out = nn.forward(X)

print("\nFinal Output after extra iterations:", final_out)
print("\nFinal weights Input→Hidden:\n", nn.weights_input_hidden)
print("\nFinal weights Hidden→Output:\n", nn.weights_hidden_output)
```

```
Streaming output truncated to the last 5000 lines.
 [[ 0.27576954  0.67713977]
 [-0.52423046  0.47713977]]

--- Forward Pass ---
Hidden linear: [[-0.49692184  2.30855908]]
Hidden activation: [[0.37826432 0.90958342]]
Output linear: [[1.05721816]]
Output activation: [[0.74215857]]

--- Backward Pass ---
Output error: [[0.75784143]]
Output delta: [[0.14501995]]
Hidden delta: [[-0.00124486  0.0140435 ]]

Updated W_hidden_output:
 [[-0.03101445]
 [ 1.19068016]]
Updated W_input_hidden:
 [[ 0.27552057  0.67994847]
 [-0.52447943  0.47994847]]

--- Forward Pass ---
Hidden linear: [[-0.49791773  2.31979388]]
Hidden activation: [[0.37803013 0.91050315]]
Output linear: [[1.07239363]]
Output activation: [[0.74505185]]

--- Backward Pass ---
Output error: [[0.75494815]]
Output delta: [[0.14340209]]
Hidden delta: [[-0.00104572  0.01391361]]

Updated W_hidden_output:
 [[-0.02559342]
 [ 1.20373696]]
Updated W_input_hidden:
 [[ 0.27531142  0.68273119]
 [-0.52468858  0.48273119]]

--- Forward Pass ---
Hidden linear: [[-0.49875431  2.33092477]]
Hidden activation: [[0.37783346 0.91140604]]
Output linear: [[1.08742308]]
Output activation: [[0.74789616]]

--- Backward Pass ---
Output error: [[0.75210384]]
Output delta: [[0.14180729]]
Hidden delta: [[-0.00085317  0.01378308]]

Updated W_hidden_output:
 [[-0.02023547]
 [ 1.21666137]]
Updated W_input_hidden:
 [[ 0.27514079  0.68548781]
 [-0.52485921  0.48548781]]
```